COMPTIA PENTEST+ · PT0-003 · OBJECTIVES V3.0

Domain 3.0 / Vulnerability Discovery & Analysis

A memorization-first study guide: pick the right scan, then prove the finding is real.

17%	~15	3	38%	3.2
of the exam	of 90 questions	objectives (3.1–3.3)	combined with Domain 2.0	the objective that decides your score

COLOUR KEY USED THROUGHOUT

STATIC examines artefacts at rest — source, images, config · DYNAMIC interacts with something running · HYBRID instrumented or credentialed: both at once

Domain 2 sorted tools by *passive vs. active*. Domain 3 sorts them by *static vs. dynamic*. Learn which column each scan sits in and half the domain answers itself.

01 The shape of the domain — and why 3.2 carries the most weight

02 The scan taxonomy — every scan type in objective 3.1, sorted

03 SAST · DAST · IAST · SCA — the four-way you must not confuse

04 Authenticated vs. unauthenticated scans

05 Containers, Kubernetes, and IaC

06 The objective 3.1 tool roster — nine tools, one line each

07 ICS and OT assessment — the safety exception

08 Validating results — false positives, false negatives, the confidence ladder

09 Scoring and prioritisation — CVSS, CVE, CWE, EPSS

10 Public exploit selection — how to vet code before you run it

11 Scripting to validate at scale

12 Objective 3.3 physical security — five techniques, five controls

13 A 12-day plan and how to practise

14 Rapid-recall drill — 38 prompts, fold-over format

15 One-page cheat sheet

01

The shape of the domain

Three objectives that behave very differently. Two are recall; one is judgement, and it's the one that decides your score.

OBJ.	TITLE	WHAT IT REALLY TESTS	QUESTION STYLE
3.1	Conduct vulnerability discovery using various techniques	Choosing the right *kind* of scan for the target and the constraint, plus tool recall	Scenario → best scan type; tool multiple choice
3.2	Analyze output from reconnaissance, scanning, and enumeration phases	**Judgement.** Is this finding real? Is the scan complete? Which exploit fits? Why did the scan fail?	Output snippet + "what now?"; PBQs
3.3	Explain physical security concepts	Five terms and the controls that stop them. Pure recall — free marks	Definition matching, one-liners

HOOK — THE DOMAIN IN ONE SENTENCE

Domain 2 finds what exists. Domain 3 decides what matters. Recon and enumeration produce claims; this domain turns claims into findings, and findings into a priority order.

Where the marks actually are

- **3.2 is the heart of it.** Every "the scanner reported X — what do you do?" question lives here, and the answer is almost always *validate before you act*.
- **3.1 rewards taxonomy, not tool trivia.** If you know which scan type applies, the tool name usually follows.
- **3.3 is nine bullet points total.** Learn them once, get them right every time. Don't skip a guaranteed-marks objective because it feels unglamorous.
- **Scoring metrics (CVSS, CVE, CWE, EPSS) are formally objective 4.1**, but they get tested through Domain 3-style analysis questions. Section 09 covers them for that reason.

02

The scan taxonomy

Objective 3.1 lists a dozen scan types. Sorting them by what they examine and what they need makes them one table instead of a dozen facts.

HOOK — FIVE QUESTIONS THAT IDENTIFY ANY SCAN

What does it look at? Does the thing have to be running? Do I need credentials? What does it find? What does it always miss? Answer those five and you have named the scan type.

SCAN TYPE	MODE	FINDS	BLIND TO
Network scan TCP/UDP, stealth	Dynamic	Live hosts, open ports, service versions, network-reachable vulnerabilities	Anything local to the host: patch level, file permissions, local config
Host-based scan	HYBRID	Missing patches, local misconfiguration, weak permissions, endpoint hardening. Agent or credentialed	How the host looks from outside; network path issues
SAST incl. IaC + source code analysis	Static	Insecure code patterns, injection sinks, hardcoded secrets, bad crypto, misconfigured Terraform/CloudFormation	Runtime behaviour, environment config, whether the flaw is actually reachable
DAST	Dynamic	Exploitable behaviour of the running app: injection, auth flaws, session handling, server config	The line of code responsible; code paths its crawler never reaches
IAST	HYBRID	Both — an agent inside the running app correlates the request with the code path	Untested functionality; needs instrumentation and test coverage
SCA	Static	Third-party and open-source components with known CVEs; licence problems; produces an SBOM	Flaws in code your team wrote
Mobile scan	HYBRID	Hardcoded keys, insecure storage, weak TLS pinning, excessive permissions in an APK/IPA	Server-side flaws behind the app. Standard: OWASP MASVS. Tool: MobSF
Container scan	Static	Vulnerable OS/library packages in image layers, embedded secrets, bad Dockerfile practice	Runtime behaviour and orchestrator configuration
Sidecar scan	Dynamic	A scanning container deployed alongside the workload in the same pod — inspects the running app without modifying its image	Nothing about the build pipeline; adds runtime overhead

Secrets scan	Static	Credentials, API keys, tokens, private keys in repos, history, images, and config	Secrets held only in memory or in a proper vault
Wireless scan SSID, channel, signal strength	Dynamic	Networks and APs in range, channel use, and how far usable signal leaks past the building	Wired-side issues. Signal strength is a *physical* security finding as much as a wireless one
ICS / OT assessment	Static	Device inventory and protocol behaviour, gathered by **manual assessment** and **port mirroring**	Anything an active scan would have told you — which you must not run. See section 07

THE DISTINCTION THAT GENERATES QUESTIONS

Static means the artefact doesn't have to be running; dynamic means it does. That's why SAST can run on every commit but DAST needs a deployed environment — and why "we need results before the app is deployed" always points at SAST or SCA.

03

SAST · DAST · IAST · SCA

The single most reliable source of wrong answers in Domain 3. Four tools, four different jobs.

	SAST	DAST	IAST	SCA
Looks at	Source code / bytecode	The running application	The running app, from inside	Dependencies and libraries
View	White box	Black box	Grey box	Inventory
Needs	Code access	A deployed URL	An agent + test traffic	A manifest or lock file
SDLC stage	Earliest — commit / build	Test / staging / production	Test, while suites run	Any — build and continuously
Strength	Points at the exact line; finds flaws before deployment	Proves real, exploitable behaviour	Low false positives; links request to code path	Catches the vulnerable component nobody wrote
Weakness	**High false positives**; no runtime context	Can't tell you where in the code; misses unreachable paths	Only covers what the tests exercise; setup cost	Says nothing about your own code
Mode	Static	Dynamic	HYBRID	Static

HOOK — FOUR NAMES, FOUR MEANINGS

SAST = **S**ource **A**t re**ST** · **DAST** = the app is **D**oing something · **IAST** = an agent **I**nside it · **SCA** = **S**omeone else's **C**ode, **A**udited.

HOW THE EXAM PHRASES EACH ONE

THE SCENARIO SAYS…	ANSWER
"Find flaws before the code is deployed", "shift left", "scan on every commit"	**SAST**
"Test the production web app", "no source code available", "black box"	**DAST**
"Reduce false positives while the QA suite runs", "confirm the finding is reachable at runtime"	**IAST**
"Identify vulnerable open-source libraries", "produce an SBOM", "supply chain"	**SCA**
"Review the Terraform before it's applied"	**SAST / IaC scanning**
"Analyse the mobile app package"	**Mobile scan** (MobSF; standard is MASVS)

04

Authenticated vs. unauthenticated scans

One line of configuration that changes almost everything about the results — and the source of most false positives and false negatives on the exam.

UNAUTHENTICATED (UNCREDENTIALED)	AUTHENTICATED (CREDENTIALED)

Point of view	An outsider with no access	A user or admin on the box
How it decides	*Infers* from banners, ports, and responses	*Reads* installed packages, patch level, registry, and config
False positives	High — banner says 2.4.29, patch was backported, version string never changed	Low — it can see the actual patch
False negatives	High — anything not exposed to the network is invisible	Much lower
Good for	Modelling what an external attacker sees; attack-surface work	Completeness, patch verification, compliance and config review
Mode	Dynamic	HYBRID

HOOK

Unauthenticated scanning guesses. Authenticated scanning knows. When a question complains about false positives, credentials are usually the fix. When it asks what an attacker would see, credentials are the wrong answer.

WHERE THE CREDENTIALS COME FROM MATTERS

Scanning credentials are an engagement decision, not a technical one — they belong in the scope and rules of engagement (objective 1.1), and they raise the question of who is responsible if a credentialed scan changes the system state. Expect at least one question that blends Domain 1 with Domain 3 this way.

05

Containers, Kubernetes, and IaC

New material in PT0-003, and easy to under-study. Learn the four layers and which scan belongs to each.

LAYER	WHAT GOES WRONG	WHAT YOU SCAN WITH
Image	Vulnerable OS and library packages, secrets baked into a layer, `:latest` tags, `USER root`	Trivy, Grype STATIC
Registry / pipeline	Unsigned or tampered images, no scan gate before deploy, IaC that provisions insecurely	Trivy (IaC + image), SCA STATIC
Orchestrator	Over-permissive RBAC, exposed dashboard or API server, privileged pods, reachable metadata service	Kube-hunter DYNAMIC
Host / runtime	Exposed Docker socket, container running as root, no resource limits, no seccomp/AppArmor	Docker Bench (CIS), sidecar scan DYNAMIC

THE LAYER TRAP WORTH MEMORISING

Deleting a file in a later image layer does **not** remove it — the earlier layer still contains it. A secret that was added and then deleted in a Dockerfile is still extractable from the image. That's why container scanning and secrets scanning are separate bullets in the objectives.

KUBE-HUNTER'S THREE MODES — A FAVOURITE QUESTION SHAPE

- **Remote** — scan the cluster from outside by hostname or IP: the external attacker's view.
- **Interface** — scan every network interface on the machine you're on.
- **Pod** — run inside the cluster as a pod: what an attacker who already has a container sees.
- **Active mode** performs actions that can change cluster state. It needs explicit authorisation in the rules of engagement.

HOOK — SBOM

Syft builds the bill of materials; Grype checks it against known vulnerabilities. An SBOM is an inventory, not a verdict — SCA is what turns the inventory into findings.

IaC scanning is SAST applied to infrastructure: Terraform, CloudFormation, ARM templates, Kubernetes manifests. It catches the public storage bucket, the `0.0.0.0/0` security group, and the unencrypted volume *before* anything is provisioned. Container escape, image tampering, and supply-chain attacks are the offensive counterparts in objective 4.6.

06

The objective 3.1 tool roster

Nine named tools. One sharp line each, plus the phrasing that names them.

TOOL	MODE	IDENTITY — AND THE QUESTION THAT NAMES IT

Nikto	Dynamic	Web *server* scanner: thousands of known dangerous files, CGIs, default installs, outdated server versions. **Extremely loud — no stealth option.** Named by "quickly check the web server for known issues"
Greenbone / OpenVAS	HYBRID	The open-source general-purpose vulnerability scanner. Feed of network vulnerability tests. Named by "open-source scanner" or "no budget"
Tenable Nessus	HYBRID	The commercial equivalent: plugin-based, strong credentialed scanning and compliance policies. Named by "enterprise vulnerability scanner"
TruffleHog	Static	Secrets scanner that walks **git history**, so committed-then-deleted keys still surface. Named by "credentials in a repository"
BloodHound	HYBRID	Graphs Active Directory relationships and computes the shortest path to Domain Admin. Collector: SharpHound. Named by "attack path" or "which account gets me to DA"
PowerSploit	HYBRID	PowerShell offensive modules. **PowerUp** finds local privilege-escalation misconfigurations; **PowerView** does AD recon. Heavily signatured by AV
Grype	Static	Container image and filesystem vulnerability scanner. Pairs with Syft for SBOMs. Named by "scan this image for vulnerable packages"
Trivy	Static	The all-rounder: images, filesystems, git repos, IaC, Kubernetes, secrets, licences. Named by "one tool for images *and* Terraform"
Kube-hunter	Dynamic	Hunts weaknesses in a Kubernetes cluster from outside, on-host, or as a pod. Named by "assess the cluster"

DISTRACTOR DEFENCE

CONFUSABLE	THE SEPARATING LINE
Nessus vs **OpenVAS**	Same job. Nessus is the commercial answer, Greenbone/OpenVAS the open-source one
Trivy vs **Grype**	Trivy is broad (images, IaC, K8s, secrets). Grype is focused on package vulnerabilities in images and filesystems
Kube-hunter vs **Docker Bench**	Kube-hunter probes a live cluster for weaknesses. Docker Bench audits a Docker host against the CIS benchmark
Nikto vs **Burp / ZAP**	Nikto checks the server for known bad content. Burp and ZAP test the application's own logic (Domain 4.5)
TruffleHog vs **SCA**	TruffleHog looks for secrets in your repo. SCA looks for vulnerabilities in your dependencies
Nmap vs a vulnerability scanner	Nmap answers "what's there?" A scanner answers "what's wrong with it?" — the boundary between Domains 2 and 3

07

ICS and OT — the safety exception

Two approved techniques, one reason, and a triad that runs backwards. Reliably worth a question.

HOOK — OT FLIPS THE TRIAD

IT protects **C-I-A**. Operational technology protects **A-I-C** — availability first, because the process is physical and stopping it can hurt someone. Every ICS answer follows from that inversion.

APPROVED TECHNIQUE	WHAT IT MEANS IN PRACTICE
Manual assessment	Review configurations, firmware versions, network diagrams, and documentation; interview operators; compare against advisories by hand
Port mirroring (SPAN)	Have the switch copy traffic to your analysis port and read it with Wireshark/tcpdump. You see the protocols and devices without ever transmitting

WHY ACTIVE SCANNING IS OUT

Legacy controllers were built for a closed, trusted network. A malformed or merely unexpected packet — including an ordinary port scan or version probe — can hang a PLC. A stopped process line is a safety event, not an IT ticket. **Any answer containing "scan", "fuzz", "brute force", or "exploit" against live OT is wrong.** If active testing is genuinely required, it happens on an offline replica or during a planned outage, documented in the rules of engagement.

CONTEXT YOU SHOULD BE ABLE TO PLACE

- **Purdue model** (objective 1.3) — Level 0 field devices and sensors, Level 1 controllers/PLCs, Level 2 supervisory HMI/SCADA, Level 3 site operations, Level 3.5 industrial DMZ, Levels 4–5 enterprise IT. The lower the level, the tighter the testing constraints.
- **Protocols** — Modbus (502), DNP3 (20000), S7comm (102), EtherNet/IP (44818), BACnet (47808/udp). Nearly all are cleartext with no authentication, which is precisely why sniffing works so well and probing is unnecessary.
- **Attacks** against these — register manipulation, Modbus and CAN bus attacks, replay — belong to objective 4.9. Domain 3 stops at assessment.

08

Validating results — objective 3.2

The most heavily tested idea in the domain: a scanner produces claims, and a pentester produces evidence.

The four outcomes

OUTCOME	MEANING	WHAT IT COSTS
True positive	Reported and real	Nothing — this is the goal. Validate it, then rank it
False positive	Reported but not real	Wasted remediation effort and lost credibility. Enough of them and the client stops reading your reports
False negative	Real but not reported	**The dangerous one.** The client believes they're secure. This is the failure that gets people breached
True negative	Correctly reported as clean	Nothing — but only trustworthy if your coverage was complete

WHY FALSE POSITIVES HAPPEN	WHY FALSE NEGATIVES HAPPEN
• **Backported patches** — the vendor fixed the bug without changing the version string, so banner-based checks still fire • Unauthenticated scanning inferring from a banner • Compensating controls already in place (WAF, segmentation, disabled module) • Plugin heuristics matching loosely • A load balancer answering for several different backends • Honeypots and tarpits deliberately answering everything	• **No credentials** — local issues stay invisible • Default port range only (top 1,000), UDP skipped, IPv6 never scanned • Host was off or filtered during the scan window • Out-of-date plugin or vulnerability feed • Scan timed out, was throttled, or the IPS blocked the scanner • Scope exclusions the client forgot to mention

The confidence ladder

Every finding sits on one of these rungs. Know which rung you're on, and never write the report as though you're a rung higher.

1	**Scanner claim**	A plugin fired. Nothing more	*"The scanner reported…" — not yet reportable as a finding*
2	**Corroborated**	A second tool, or an authenticated re-check, agrees	*"Two independent checks agree"*
3	**Manually verified**	You observed the vulnerable condition yourself — the version, the config, the response	*"Confirmed present" — the minimum bar for most report findings*
4	**Proven exploitable**	You produced the expected effect, safely and within scope	*"Exploited successfully" — with the evidence attached*
5	**Impact demonstrated**	You showed what it means for the business: data reached, access gained, control lost	*The attack narrative and business impact of objectives 1.4 and 1.5*

> **HOOK**
> **A scanner reports; a pentester proves.** When a question gives you a scanner result and asks what to do next, the answer is the next rung up the ladder — not remediation advice, and not an exploit fired blind at production.

Scan completeness — the pre-report checklist

- Did I cover the whole authorised scope, and only the authorised scope?
- All 65,535 TCP ports, or just the default 1,000? Was UDP scanned at all? IPv6?
- Were any hosts down, filtered, or newly built during the window — and can I rescan them?
- Were credentials used where they were available? Which systems lacked them?
- Is the plugin or vulnerability feed current?
- Did the scan finish, or did it time out, get rate-limited, or get blocked mid-run?
- Do two tools disagree anywhere — and have I resolved which is right?

Troubleshooting scan configurations

SYMPTOM	CHECK FIRST
No hosts found at all	Host discovery blocking ICMP — rerun with host discovery disabled; wrong subnet or VLAN; no route

Hosts found, but zero vulnerabilities	Plugin families disabled, wrong scan policy or template, feed never updated
Results are far thinner than expected	Credentials failed silently — check the authentication log before trusting a clean result
Scan is far slower than expected or dies	Rate limiting, an IPS blocking the scanner's address, or timing set too aggressive for the link
Every port reports open	A tarpit, a load balancer, or an inline security device answering on the host's behalf
Findings the client insists are already patched	Backported patch — verify the package version on the host, not the service banner

09

Scoring and prioritisation

Formally objective 4.1, but tested through Domain 3 analysis questions. Learn the four acronyms and how they combine.

ACRONYM	ANSWERS THE QUESTION	LOOKS LIKE
CVE	*Which* vulnerability is this? A unique identifier for one specific flaw in one product	CVE-2021-44228
CWE	*What kind* of weakness is it? The class or root cause, reused across thousands of CVEs	CWE-79 (XSS), CWE-89 (SQL injection), CWE-22 (path traversal)
CVSS	*How severe* would exploitation be?	0.0–10.0, plus a metric vector
EPSS	*How likely* is it to be exploited in the wild in the next 30 days?	A probability, 0 to 1

HOOK — THE FOUR ACRONYMS AS A SENTENCE

CWE is the disease, CVE is the case, CVSS is how bad the case is, EPSS is how likely you are to catch it.

CVSS BASE METRICS — THE EIGHT YOU SHOULD RECOGNISE

METRIC	VALUES
Attack Vector (AV)	Network · Adjacent · Local · Physical — *Network scores highest*
Attack Complexity (AC)	Low · High
Privileges Required (PR)	None · Low · High — *None scores highest*
User Interaction (UI)	None · Required
Scope (S)	Unchanged · Changed — *Changed means it escapes its security boundary*
Confidentiality / Integrity / Availability	High · Low · None, each rated separately

Severity bands: 0.1–3.9 Low · 4.0–6.9 Medium · 7.0–8.9 High · 9.0–10.0 Critical. Base scores can be adjusted by *temporal* metrics (is there exploit code? a patch?) and *environmental* metrics (how much does this asset matter here?).

THE POINT COMPTIA WANTS YOU TO MAKE

CVSS measures severity, not risk. A 9.8 on an isolated lab box with no data matters less than a 6.5 on the internet-facing system that holds cardholder data. Risk needs severity *plus* exploit likelihood *plus* business context — which is why the report has a business impact analysis (1.2) and not just a list of scores.

HOW TO RANK IN PRACTICE

Sort by all four dimensions, in this order of usefulness:

1. **Is it exploitable right now?** Public exploit code available, and EPSS high.
2. **How bad if it is?** CVSS, adjusted for what the asset actually does.
3. **What does the asset hold or control?** High-value asset identification (4.1).
4. **How cheap is the fix?** A one-line config change that kills a Medium beats a six-month re-architecture that kills a High.

Beyond the objectives but worth knowing: **CISA's Known Exploited Vulnerabilities (KEV) catalogue** is the practitioner's shortcut — if it's on the KEV list, it is being exploited today, and it goes to the top regardless of its CVSS score.

10

Public exploit selection

Named explicitly in objective 3.2. The exam tests judgement about code you didn't write.

SOURCE	WHAT YOU GET
Exploit-DB / `searchsploit`	Archived proof-of-concept code, offline and searchable from Kali
Metasploit modules	Maintained, parameterised exploits with a reliability rank and swappable payloads
GitHub proof-of-concept	The newest code for the newest CVEs — and the least trustworthy
Vendor advisories / NVD references	Authoritative affected-version lists and mitigations. Start here to confirm applicability

```
searchsploit apache 2.4.49          # search by product and version
searchsploit -x 50383               # examine the code before running anything
searchsploit -m 50383               # mirror a copy into the current directory
searchsploit --nmap scan.xml        # correlate an Nmap XML scan straight to candidate exploits
```

The eight-point vetting checklist

CHECK	ASK YOURSELF
1. Version match	Does it target the *exact* version and build you found — including the patch level?
2. Platform match	Right OS, distribution, architecture (x86 vs x64), and language runtime?
3. Preconditions	Does it need credentials, a specific module enabled, a non-default config, or local access?
4. **Read the code**	Does it do only what it claims? Look for obfuscated blobs, outbound connections to unknown hosts, or destructive commands
5. Payload	Is the shellcode hardcoded for someone else's listener? Regenerate it with `msfvenom`
6. Reliability	Metasploit ranks modules *excellent* → *great* → *good* → *normal* → *average* → *low* → *manual*. Prefer the top of that list on production
7. Blast radius	Is this actually a denial of service dressed up as code execution? Could it crash the service or corrupt data?
8. Authorisation	Do the rules of engagement permit exploitation of this host, in this window, with this level of risk?

THE RULE THE EXAM WANTS BACK FROM YOU

Test it in a lab replica before you run it against the client. Unvetted public exploit code is the classic way a penetration test becomes an outage — and an unreviewed script from a stranger's repository is an obvious supply-chain risk to your own toolkit.

HOOK

Match the version, read the code, replace the payload, test in the lab. Four steps, in that order, every time.

11

Scripting to validate at scale

Objective 3.2 says "use scripting to validate results." That means turning hundreds of scanner rows into a handful of confirmed findings.

A scanner hands you 400 rows across 60 hosts. You cannot manually verify all of them, and you cannot report them unverified. Scripting is the bridge: re-check one specific fact on every affected host, then keep only the hosts that fail.

```
# BASH — re-check the banner on every host the scanner flagged
while read ip; do
  ver=$(curl -sI --max-time 3 "http://$ip" | grep -i '^Server:' | tr -d '\r')
  echo "$ip -> ${ver:-no response}"
done < flagged_hosts.txt | tee validated.txt
```

```
# BASH — pull just the hosts with 445 open out of an Nmap greppable file
grep "445/open" scan.gnmap | cut -d' ' -f2 | sort -u > smb_hosts.txt
wc -l smb_hosts.txt        # the number that goes in the report
```

```python
# PYTHON — confirm a version string rather than trusting the plugin
import requests
for ip in open("flagged_hosts.txt"):
    ip = ip.strip()
    try:
        r = requests.get(f"http://{ip}", timeout=3)
        server = r.headers.get("Server", "unknown")
        if "2.4.49" in server:
            print(f"{ip} CONFIRMED {server}")
    except requests.RequestException:
        print(f"{ip} unreachable")
```

```powershell
# POWERSHELL — the authenticated answer: is the patch actually installed?
Get-HotFix -Id KB5005565 -ComputerName (Get-Content hosts.txt) -ErrorAction SilentlyContinue
```

VALIDATION JOB	THE SCRIPTED MOVE
Confirm a version-based finding	Re-request the banner or header on each host and match the string yourself
Confirm a patch-based finding	Query the host for the package or KB — the authoritative check, and it needs credentials
Reduce a scan file to a target list	`grep` + `cut` + `sort -u` on the greppable output, or parse the XML
Count instances for the report	`sort -u | wc -l` — "17 of 60 hosts affected" is a far stronger finding than "affected hosts"
Cross-check two scanners	Normalise both outputs to `host,port,finding` and diff them; investigate anything only one tool saw

HOOK

Scanners produce rows; scripts produce findings. Every 3.2 scripting question is really asking: how do you check one fact on many hosts without doing it by hand?

12

Objective 3.3 — physical security

Five techniques, five controls, nine bullet points in total. The cheapest marks in the domain.

TECHNIQUE	HOW IT'S DONE	WHAT IT PROVES	CONTROL THAT STOPS IT
Tailgating	Follow an authorised person through a controlled door — hands full, on the phone, carrying coffee	Door control depends on staff behaviour, not just hardware	Access control vestibule (mantrap), turnstiles, awareness training
Site survey	Observe and document: entrances, badge readers, camera coverage and blind spots, guard patterns, smoking areas, loading docks, fence lines, dumpsters, wireless signal leaking outside	Everything else. It's the reconnaissance that makes the other four possible	Lighting, signage, camera placement, perimeter design
USB drops	Leave prepared drives in car parks, lobbies, and restrooms and see who plugs one in. Payload beacons back or emulates a keyboard	User behaviour and endpoint device policy. Report the plug-in *rate*	Disable autorun, USB device control, user training
Badge cloning	Read a credential at close range and write a duplicate. 125 kHz proximity cards clone trivially; 13.56 MHz smart cards with mutual authentication are far harder	The credential technology is outdated	Encrypted high-frequency credentials, biometrics, MFA at the door, photo checks
Lock picking	Picks and rakes on pin-tumbler locks; bypass tools (shove knife, under-door tool); shimming padlocks	Mechanical security is weaker than the electronic layer people focus on	High-security cores, restricted keyways, bypass-resistant hardware, door alarm contacts

HOOK — THE SEQUENCE

Survey first, then choose your door. Site survey is reconnaissance (Domain 2's logic applied to a building); the other four are the exploitation step. If a question asks what comes first, it's the survey.

RISK TO THE PENETRATION TESTER — OBJECTIVE 1.1, TESTED HERE

Physical testing is the one place a pentester can be detained or hurt. So: carry the **signed authorisation letter on your person**, know the name and mobile number of your escalation contact, agree in advance what happens if you're challenged, and confirm the legality of your tools — **possessing lock picks is an offence in some jurisdictions**, which is exactly why the objectives' hardware list says "lock pick kit (where applicable)."

The controls in the right-hand column above come from objective 1.5's *physical controls*: access control vestibule, biometric controls, and video surveillance. Expect at least one question that hands you a physical finding and asks for the remediation — that's a 3.3 finding answered with a 1.5 control.

13

A 12-day plan, and how to practise

Domain 3 is smaller than Domain 2 but sits directly on top of it. Study it immediately after, while the scan output is still fresh.

DAYS	FOCUS	DONE WHEN YOU CAN…
1–2	The scan taxonomy (section 02)	Name every scan type in 3.1 with its mode, what it finds, and what it misses
3	SAST · DAST · IAST · SCA	Answer all six scenario phrasings in section 03 without hesitating
4	Authenticated vs. unauthenticated	Explain why credentials cut false positives, and when they're the wrong choice
5	Containers, Kubernetes, IaC	Match each of the four layers to its scan, and recite Kube-hunter's three modes
6	The nine-tool roster	Give the one-line identity for each, cold, and separate every confusable pair
7	ICS / OT	Justify the two approved techniques from the inverted triad, unprompted
8–9	Validation: FP/FN, completeness, troubleshooting	List six causes of false negatives and diagnose all six symptoms in section 08
10	CVSS, CVE, CWE, EPSS	Rank three findings and defend the order using severity, likelihood, and asset value
11	Exploit selection + validation scripting	Walk the eight-point checklist, and write one loop that re-checks a banner across a host list
12	Physical security + mixed practice	Name all five techniques with their controls, then work through unfamiliar scan output end to end

Four practice habits

1. **Scan the same host twice — once without credentials, once with.** Diff the two reports. That single exercise teaches authenticated vs. unauthenticated, false negatives, and scan completeness at the same time, and you'll never forget which one found more.
2. **Chase one finding all the way up the ladder.** Take a single scanner result and get it to rung 3, then rung 4, writing down what evidence each rung required. Do it once properly and the whole of 3.2 makes sense.
3. **Deliberately produce a false positive.** Scan a host whose service you've patched but whose banner still shows the old version. Seeing your scanner confidently wrong is the most durable lesson in the domain.
4. **Read exploit code you don't intend to run.** Open five Exploit-DB entries and work out what each requires and what it would break. That's exactly the judgement objective 3.2 tests.

A lab that's enough

- **Greenbone/OpenVAS** (free) against **Metasploitable** — an intentionally vulnerable target gives you real findings to validate rather than an empty report.
- **Trivy** or **Grype** against any public image, then against a deliberately old one such as an unpatched base image: `trivy image ubuntu:18.04`.
- **TruffleHog** against a throwaway repo where you commit a fake key, then delete it — proving to yourself that it's still in the history.
- An intentionally vulnerable web app (OWASP Juice Shop, DVWA) for DAST practice with ZAP.

SCOPE DISCIPLINE

Scan only what you own or are authorised in writing to test, and never point a vulnerability scanner at equipment you can't afford to reboot. The habit the exam rewards is the habit that keeps you employable.

14

Rapid-recall drill

Cover the right column. Say the answer out loud before you look. Anything you miss twice goes on an index card.

SAST and DAST in three words each?	Source code at rest / running application, from outside
Which scan finds vulnerable third-party libraries?	SCA — software composition analysis

Which scan type produces an SBOM?	SCA. Syft builds the inventory, Grype checks it for vulnerabilities
Which application scan has the highest false-positive rate?	SAST — it has no runtime context to rule findings out
Which one can't tell you the line of code?	DAST — it only sees the application's behaviour
What does IAST add?	An agent inside the running app, correlating a live request with the code path. Fewer false positives
"Review the Terraform before it's applied"?	SAST applied to infrastructure as code
The two approved ICS assessment techniques?	Manual assessment and port mirroring
How does OT order the triad?	Availability, integrity, confidentiality — the reverse of IT
What does credentialed scanning mainly buy you?	Accuracy: far fewer false positives and far fewer false negatives
Which scan outcome is the dangerous one?	The false negative — the client believes they're secure
Classic false-positive cause on enterprise Linux?	A backported patch: fixed, but the version string never changed
Name four causes of false negatives.	No credentials · default port range, UDP or IPv6 skipped · host down or filtered during the window · stale plugin feed
What is Nikto?	A very loud web *server* scanner: known dangerous files, CGIs, default installs, outdated versions
Open-source counterpart to Nessus?	Greenbone / OpenVAS
Which tool finds secrets in deleted commits?	TruffleHog — it walks git history
Trivy vs Grype?	Trivy is broad — images, filesystems, repos, IaC, Kubernetes, secrets. Grype focuses on package vulnerabilities
Kube-hunter's three modes?	Remote, interface, and pod — outsider, on-host, and inside-the-cluster views
What does Docker Bench check?	A Docker host's configuration against the CIS benchmark
Why doesn't deleting a file in a Dockerfile remove the secret?	The earlier image layer still contains it and can be extracted
PowerSploit's two named modules?	PowerUp (local privilege-escalation misconfigurations) and PowerView (AD recon)
BloodHound's collector?	SharpHound
CVE vs CWE?	CVE is the specific case; CWE is the class of weakness behind it
What does EPSS estimate?	The probability the vulnerability will be exploited in the wild in the next 30 days
CVSS 7.0–8.9 is which band?	High. 9.0+ is Critical, 4.0–6.9 Medium, 0.1–3.9 Low
Which Attack Vector value scores highest?	Network
CVSS Scope: Changed means?	The impact escapes the vulnerable component's security boundary
Is CVSS a measure of risk?	No — severity only. Risk needs likelihood and business context added
Rung 3 of the confidence ladder?	Manually verified — you observed the vulnerable condition yourself
A scanner reports a critical finding. Next step?	Validate it — move one rung up the ladder, not straight to exploitation or remediation advice
First thing to do with a GitHub proof of concept?	Read the code, after checking the version and platform match
Metasploit's highest reliability rank?	Excellent
searchsploit flag that correlates an Nmap scan?	`searchsploit --nmap scan.xml`
Which physical test measures user behaviour?	The USB drop — report the plug-in rate
Control that stops tailgating?	An access control vestibule, plus awareness training
Which badge technology clones trivially?	125 kHz low-frequency proximity cards
What do you carry on a physical engagement?	The signed authorisation letter, and your escalation contact's number
Every port reports open. Why?	A tarpit, load balancer, or inline security device answering on the host's behalf

15

One-page cheat sheet

Print this page on its own. It's the last thing to read before the exam.

THE FOUR APPLICATION SCANS

- **SAST** — source at rest; earliest; high false positives
- **DAST** — running app, black box; proves behaviour
- **IAST** — agent inside; links request to code
- **SCA** — dependencies and SBOM; supply chain

SCAN → MODE

- **Static:** SAST, IaC, SCA, container image, secrets, ICS review
- **Dynamic:** network, DAST, sidecar, wireless, Kube-hunter
- **Hybrid:** IAST, host-based, credentialed, mobile

CONFIDENCE LADDER

- 1 scanner claim → 2 corroborated → 3 manually verified → 4 proven exploitable → 5 impact demonstrated
- Rung 3 is the minimum bar for a report finding

FALSE POSITIVE VS. FALSE NEGATIVE

- **FP causes:** backported patch · unauthenticated inference · compensating control already present
- **FN causes:** no credentials · default port range, UDP/IPv6 skipped · stale feed or blocked scan
- FP costs credibility; **FN costs the client**

EVERY MNEMONIC IN THIS GUIDE

- **Static = at rest · Dynamic = running**
- **S**ource **A**t re**ST** / **D**oing / **I**nside / **S**omeone else's **C**ode **A**udited
- **OT flips the triad** — A-I-C, availability first
- **Unauthenticated guesses; authenticated knows**
- **A scanner reports; a pentester proves**
- **CWE disease · CVE case · CVSS how bad · EPSS how likely**
- **Match the version, read the code, replace the payload, test in the lab**
- **Scanners produce rows; scripts produce findings**
- **Survey first, then choose your door**

CUE WORD → ANSWER

- "before deployment", "shift left", "every commit" ⇒ **SAST**
- "no source code", "test production" ⇒ **DAST**
- "vulnerable libraries", "SBOM" ⇒ **SCA**
- "fewer false positives at runtime" ⇒ **IAST**
- "too many false positives" ⇒ **credentialed scan**
- "what an external attacker sees" ⇒ **unauthenticated**
- "live PLC / SCADA" ⇒ **manual + port mirroring**
- "scan the image" ⇒ **Trivy / Grype**
- "assess the cluster" ⇒ **Kube-hunter**
- "secrets in the repo" ⇒ **TruffleHog**
- "web server, quickly, loudly" ⇒ **Nikto**
- "path to Domain Admin" ⇒ **BloodHound**
- "scanner says critical — now what?" ⇒ **validate it**
- "which one first?" ⇒ **exploitable + high value, not top CVSS**

TOOL ONE-WORD IDS

- Nikto *web server* · OpenVAS *open source* · Nessus *commercial*
- Trivy *everything* · Grype *packages* · Kube-hunter *cluster* · Docker Bench *CIS host*
- TruffleHog *secrets* · BloodHound *paths* · PowerSploit *Windows privesc*
- Syft *SBOM* · MobSF *mobile*

SCORING

- Bands: **0.1–3.9** Low · **4.0–6.9** Medium · **7.0–8.9** High · **9.0+** Critical
- Base metrics: AV · AC · PR · UI · Scope · C/I/A
- Highest-scoring values: AV Network, PR None, UI None, Scope Changed
- Severity ≠ risk. Add EPSS likelihood and asset value

PHYSICAL SECURITY 3.3

- Tailgating ⇒ access control vestibule + training
- Site survey ⇒ the reconnaissance step; do it first
- USB drops ⇒ disable autorun, device control
- Badge cloning ⇒ encrypted high-frequency credentials, biometrics
- Lock picking ⇒ high-security cores; check local law first

DON'T LOSE EASY MARKS ON

- SAST vs DAST vs IAST vs SCA
- Never actively scan live OT
- A false negative is worse than a false positive
- CVSS is severity, not risk
- Read exploit code before running it; lab first
- 3.3 is nine bullet points — don't leave them unlearned

PenTest+ PT0-003 · Domain 3.0 study guide. Objective structure, tool names, and terminology follow CompTIA's *PenTest+ PT0-003 Certification Exam Objectives, version 3.0*; the groupings, tables, mnemonics, confidence ladder, and study plan are study aids built around them, not CompTIA material. Sections 09 and parts of 05 reach into objectives 4.1 and 4.6 where the analysis work genuinely needs them. Companion to the Domain 2.0 guide — scan only systems you own or are authorised in writing to test.